Zadanie: „Zguba” — szkolny system rzeczy znalezionych

Zaprojektuj i wykonaj aplikację desktopową w **Pythonie**. Interfejs przygotuj w **Qt Designerze**, a obsługę programu zaimplementuj w Pythonie. Aplikacja ma pomagać w kojarzeniu zgłoszeń o zagubionych i znalezionych przedmiotach.

1. Opis działania

Użytkownik może dodać zgłoszenie, wybierając, czy przedmiot **zgubił**, czy **znalazł**. Podaje kategorię, kolor, miejsce oraz datę zdarzenia.

Program zapisuje zgłoszenia, wyświetla je na liście i pozwala je filtrować. Po wskazaniu zgłoszenia aplikacja wyszukuje pasujące zgłoszenia przeciwnego typu, np. do przedmiotu znalezionego szuka zgłoszeń o przedmiotach zgubionych.

Jeżeli użytkownik potwierdzi dopasowanie, oba zgłoszenia zostają oznaczone jako **rozwiązane**. Dane mają być zachowane po zamknięciu i ponownym uruchomieniu programu.

2. Opis GUI

Utwórz okno główne o tytule **„Zguba — szkolny system rzeczy znalezionych”**. Ustaw jego minimalny rozmiar na około **900 × 600 px**. W oknie umieść dwa widżety QTabWidget z zakładkami: **Dodaj zgłoszenie** oraz **Lista i dopasowania**.

Zakładka „Dodaj zgłoszenie”

Umieść w niej formularz:

Widżet	objectName	Właściwości i zastosowanie
QComboBox	cmbTyp	Wybór: „Zgubiono” albo „Znaleziono”.
QComboBox	cmbKategoria	Wybór kategorii: elektronika, klucze, odzież, dokumenty, przybory szkolne, inne.
QLineEdit	txtKolor	Kolor przedmiotu; dodaj podpowiedź „np. czarny”.
QLineEdit	txtMiejsce	Miejsce zdarzenia; dodaj podpowiedź „np. sala 12”.
QDateEdit	deData	Data zgubienia lub znalezienia; włącz kalendarz (calendarPopup) i format dd.MM.yyyy.
QPushButton	btnDodaj	Tekst: „Dodaj zgłoszenie”.
QLabel	lblKomunikat	Komunikaty o błędach i poprawnym dodaniu zgłoszenia.

Zakładka „Lista i dopasowania”

Umieść w niej:

Widżet	objectName	Właściwości i zastosowanie
QComboBox	cmbFiltrTyp	Filtrowanie: wszystkie, zgubiono, znaleziono.
QComboBox	cmbFiltrKategoria	Filtrowanie według kategorii.
QComboBox	cmbFiltrStatus	Filtrowanie: wszystkie, otwarte, rozwiązane.
QPushButton	btnFiltruj	Zastosowanie wybranych filtrów.
QPushButton	btnWyczyscFiltry	Przywrócenie widoku wszystkich zgłoszeń.
QTableWidget	tblZgloszenia	Tabela zgłoszeń z kolumnami: ID, typ, kategoria, kolor, miejsce, data, status. Ustaw tabelę jako nieedytowalną i włącz zaznaczanie całych wierszy.
QPushButton	btnSzukajDopasowan	Wyszukiwanie dopasowań dla zaznaczonego zgłoszenia.
QListWidget	lstDopasowania	Wyświetlanie znalezionych propozycji i liczby zdobytych punktów.
QPushButton	btnZamknijDopasowanie	Oznaczenie wybranego dopasowania jako rozwiązane.
QLabel	lblBrakDopasowan	Informacja, gdy nie znaleziono pasujących zgłoszeń.

Zastosuj układy Qt, np. QVBoxLayout, QHBoxLayout i QFormLayout. Nie rozmieszczaj widżetów za pomocą ręcznie wpisanych współrzędnych. Interfejs zaprojektuj w Qt Designerze, a sygnały i zdarzenia połącz z kodem w Pythonie.

3. Opis mechaniki i programowania obiektowego

Dane zgłoszenia

Każde zgłoszenie powinno zawierać:

- unikalny numer id;
- typ: zgubiono albo znaleziono;
- kategorię;
- kolor;
- miejsce;
- datę;
- status: otwarte albo rozwiązane.

Wyszukiwanie dopasowań

Dopasowuj wyłącznie zgłoszenia:

- przeciwnego typu, np. zgubiono–znaleziono;
- o tej samej kategorii;
- ze statusem „otwarte”.

Przyznawaj punkty:

- zgodna kategoria — **3 pkt**;
- zgodny kolor — **2 pkt**;

- zgodne miejsce — **1 pkt**;
- daty oddalone od siebie najwyżej o 7 dni — **2 pkt**.

Porównując tekst, ignoruj wielkość liter i zbędne spacje. Pokaż maksymalnie trzy zgłoszenia, które zdobyły co najmniej **5 punktów**. Posortuj je od najwyższego wyniku.

Po potwierdzeniu dopasowania oba zgłoszenia zmieniają status na „rozwiązane” i nie pojawiają się w kolejnych wyszukiwaniach.

Podział programu na klasy

- **Zgloszenie** — przechowuje dane jednego zgłoszenia i udostępnia metody zamiany danych na słownik oraz tworzenia obiektu ze słownika.
- **RepozytoriumZgloszen** — dodaje, wczytuje, zapisuje i filtruje zgłoszenia. Dane przechowuj w pliku zgloszenia.json w kodowaniu UTF-8(można również w pliku txt bądź liście Pythona). Jeśli plik nie istnieje, program powinien rozpocząć pracę z pustą listą.
- **Dopasowywacz** — oblicza punktację i wyszukuje najlepsze dopasowania. Ta klasa nie powinna zależeć od widżetów Qt.
- **MainWindow** — obsługuje okno, reakcje na kliknięcia, walidację formularza i odświeżanie widoków. Nie umieszczaj w niej logiki zapisu JSON ani algorytmu dopasowywania.

Walidacja i testowanie

Nie dodawaj zgłoszenia, jeśli nie wybrano kategorii albo pola koloru i miejsca są puste. Wyświetl wtedy komunikat w lblKomunikat.

Przetestuj co najmniej następujące przypadki:

1. Dodanie poprawnego zgłoszenia i jego odczyt po ponownym uruchomieniu programu.
2. Zablokowanie dodania zgłoszenia z pustym kolorem lub miejscem.
3. Przyznanie punktów za zgłoszenia z tą samą kategorią, kolorem i miejscem.
4. Uwzględnienie różnicy dat wynoszącej dokładnie 7 dni i odrzucenie różnicy 8 dni.
5. Pominięcie zgłoszeń już rozwiązanych.

4. Podsumowanie i rezultat

Aplikacja ma umożliwiać dodawanie, przeglądanie, filtrowanie i dopasowywanie zgłoszeń oraz oznaczanie spraw jako rozwiązanych. Interfejs przygotuj w Qt Designerze, logikę w Pythonie, a dane zapisuj w JSON (można używać innego typu listy w pytonie, pliki txt.)

Oddaj pliki projektu, w tym plik interfejsu .ui, pliki źródłowe .py, plik zgloszenia.json z przykładowymi danymi oraz krótką listę wykonanych testów.